TOP 50 SPRING BOOT INTERVIEW QUESTIONS

Complete Explanations & Hands-On Deep Dive Guide

PREPARE • PRACTICE • SUCCEED

Table of Contents

1. Spring Boot Fundamentals (Q1–5)
2. Core Spring (Q6–10)
3. Spring MVC (Q11–14)
4. Spring Data JPA (Q15–19)
5. Hibernate (Q20–23)
6. Spring Security (Q24–28)
7. REST APIs (Q29–33)
8. JWT (Q34–37)
9. Microservices (Q38–41)
10. Exception Handling (Q42–44)
11. Transactions (Q45–47)
12. Caching (Q48–50)

Chapter 1 — Spring Boot Fundamentals

1 What is Spring Boot and why is it used?

EXPLANATION

Spring Boot is an opinionated extension of the Spring Framework that removes the boilerplate needed to configure a Spring application. Plain Spring requires manually wiring beans, configuring a servlet container, and managing dependency versions. Spring Boot solves this with three pillars:

- **Auto-configuration** — sensibly configures beans based on what's on the classpath.
- **Starter dependencies** — curated dependency bundles (e.g. `spring-boot-starter-web`) with compatible versions.
- **Embedded servers** — Tomcat/Jetty/Undertow bundled in, so you run a JAR directly instead of deploying a WAR to an external server.

It's used because it drastically cuts time-to-first-endpoint, standardizes project structure across teams, and is production-ready via Actuator, externalized config, and metrics out of the box.

HANDS-ON

```
// Minimal Spring Boot app - the entire bootstrap
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}

@RestController
class HelloController {
    @GetMapping("/hello")
    String hello() { return "Hello, Spring Boot!"; }
}
```

Run with `mvn spring-boot:run` or `java -jar app.jar` — no external Tomcat install needed. Visit `http://localhost:8080/hello`.

> Interview angle: contrast this with plain Spring, where you'd need `web.xml`, a `DispatcherServlet` registration, and an external servlet container.

2 What are the main advantages of Spring Boot?

EXPLANATION

- **Rapid setup** via Spring Initializr and starters.
- **No XML configuration** — Java-based/annotation-based config only.
- **Embedded server** — deploy as a self-contained executable JAR.
- **Production-ready features** — health checks, metrics, and environment info via Actuator.
- **Auto-configuration** reduces boilerplate bean definitions.
- **Externalized configuration** — same artifact, different `application-{profile}.properties` per environment.

HANDS-ON

```
# build a self-contained jar
mvn clean package
java -jar target/demo-0.0.1-SNAPSHOT.jar

# check it's production ready
curl http://localhost:8080/actuator/health
# {"status":"UP"}
```

> Interview angle: be ready to name the trade-off — auto-configuration can feel "magic" and hide what's really happening, which is why `--debug` and the auto-configuration report matter (see Q3).

3 What is auto-configuration in Spring Boot?

EXPLANATION

Auto-configuration is Spring Boot's mechanism for automatically creating beans based on classpath contents, existing beans, and property values. It's driven by classes annotated `@AutoConfiguration` (previously listed in `spring.factories`, now in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`), guarded by conditional annotations like `@ConditionalOnClass`, `@ConditionalOnMissingBean`, and `@ConditionalOnProperty`.

For example, if `spring-boot-starter-data-jpa` and an H2 driver are on the classpath, Spring Boot auto-configures a `DataSource`, an `EntityManagerFactory`, and a `TransactionManager` — without you writing a single bean method.

HANDS-ON

```
@SpringBootApplication
// equivalent to:
// @Configuration
// @EnableAutoConfiguration
// @ComponentScan
public class DemoApplication { }

// See exactly which auto-configs fired and why
// run with: java -jar app.jar --debug
// Output includes a "Positive matches" / "Negative matches" report

// Excluding a specific auto-configuration
@SpringBootApplication(exclude = { DataSourceAutoConfiguration.class })
public class DemoApplication { }
```

> Interview angle: mention `@ConditionalOnMissingBean` — it lets your own `@Bean` override Boot's default, which is why auto-configuration never "fights" your explicit config.

4 What is the purpose of starter dependencies?

EXPLANATION

Starters are dependency descriptors that bundle everything typically needed for a feature, with versions aligned by the Spring Boot BOM (Bill of Materials) so you never hunt for compatible versions manually. `spring-boot-starter-web` pulls in Spring MVC, Jackson, and an embedded Tomcat in one line.

HANDS-ON

```
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

Run `mvn dependency:tree` to see exactly what each starter transitively pulled in.

5 What is Spring Boot Actuator used for?

EXPLANATION

Actuator exposes production-grade monitoring and management endpoints — health, metrics, environment properties, active beans, thread dumps — over HTTP or JMX, without you writing that infrastructure yourself. It's the backbone of observability for Boot apps in Kubernetes/cloud environments (readiness/liveness probes commonly hit `/actuator/health`).

HANDS-ON

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

# application.properties
management.endpoints.web.exposure.include=health,info,metrics,env
management.endpoint.health.show-details=always
```

```
curl http://localhost:8080/actuator/health
curl http://localhost:8080/actuator/metrics/jvm.memory.used
curl http://localhost:8080/actuator/env
```

Interview angle: mention that in production you should NOT expose all endpoints by default (e.g. `/actuator/env` can leak secrets) — restrict via `management.endpoints.web.exposure.include` and secure them behind Spring Security.

Chapter 2 — Core Spring

6 What is dependency injection?

EXPLANATION

Dependency Injection (DI) is a design pattern where an object's dependencies are supplied by an external entity (the Spring IoC container) rather than the object creating them itself. This inverts control of object creation, enabling loose coupling, easier testing (you can inject mocks), and centralized lifecycle management.

HANDS-ON

```
public interface PaymentGateway {
    void charge(double amount);
}

@Component
public class StripeGateway implements PaymentGateway {
    public void charge(double amount) {
        System.out.println("Charging $" + amount + " via Stripe");
    }
}

@Service
public class OrderService {
    private final PaymentGateway gateway; // dependency, not "new StripeGateway()"

    // Constructor injection - Spring supplies the implementation
    public OrderService(PaymentGateway gateway) {
        this.gateway = gateway;
    }

    public void checkout(double amount) {
        gateway.charge(amount);
    }
}
```

In a test, you inject a mock instead: `new OrderService(mock(PaymentGateway.class))` — no Spring container required.

7 Difference between @Component, @Service, and @Repository?

EXPLANATION

All three are stereotype annotations built on `@Component`, meaning Spring's component scan detects and registers them as beans identically. The difference is semantic and behavioral:

Annotation	Layer	Special behavior
`@Component`	Generic	None — generic bean marker
`@Service`	Business/service layer	None extra — purely semantic
`@Repository`	Persistence/DAO layer	Enables **exception translation**: platform-specific DB exceptions are converted to Spring's unchecked `DataAccessException` hierarchy

HANDS-ON

```
@Repository
public class UserRepositoryImpl {
    @PersistenceContext
    private EntityManager em;

    public User findById(Long id) {
        // A raw SQLException/JDBC exception thrown here is automatically
        // translated into a DataAccessException subtype, e.g.
        // EmptyResultDataAccessException, thanks to @Repository
        return em.find(User.class, id);
    }
}
```

8 What is the Spring Bean lifecycle?

EXPLANATION

A managed bean passes through: (1) instantiation, (2) property/dependency population, (3) `BeanNameAware` / `BeanFactoryAware` callbacks, (4) `BeanPostProcessor.postProcessBeforeInitialization`, (5) `@PostConstruct` / `InitializingBean.afterPropertiesSet()`, (6) `BeanPostProcessor.postProcessAfterInitialization`, (7) bean is ready for use, (8) on context shutdown: `@PreDestroy` / `DisposableBean.destroy()`.

HANDS-ON

```
@Component
public class ConnectionPool implements InitializingBean, DisposableBean {

    @PostConstruct
    public void init() {
        System.out.println("1. @PostConstruct - open connections");
    }

    @Override
    public void afterPropertiesSet() {
        System.out.println("2. InitializingBean - final setup checks");
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("3. @PreDestroy - closing connections");
    }

    @Override
    public void destroy() {
        System.out.println("4. DisposableBean - released resources");
    }
}
```

> Interview angle: prefer `@PostConstruct` / `@PreDestroy` over implementing the interfaces — they avoid coupling your class to Spring APIs.

9 What is Inversion of Control (IoC)?

EXPLANATION

IoC is the broader principle behind DI: control over object creation and wiring is inverted from the application code to a container/framework. Instead of your code calling `new` and orchestrating collaborators, the **ApplicationContext** (the IoC container) reads bean definitions, instantiates objects, and wires them together. DI is simply the specific technique Spring uses to implement IoC.

HANDS-ON

```
public static void main(String[] args) {
    ApplicationContext ctx =
        SpringApplication.run(DemoApplication.class, args);

    // The container owns creation & wiring — you just ask for beans
    OrderService service = ctx.getBean(OrderService.class);
    service.checkout(99.99);
}
```

Without IoC, `OrderService` would have to construct `new StripeGateway()` itself, hard-coding the dependency.

10 Constructor injection vs field injection?

EXPLANATION

Aspect	Constructor Injection	Field Injection
Immutability	Fields can be `final`	Fields cannot be `final`
Testability	Easy — plain `new` with mocks	Needs reflection or Spring test context
Circular deps	Fails fast at startup	Can silently "work" via proxies, hiding design issues
Null-safety	Guaranteed non-null once constructed	Object can exist briefly with null dependency

HANDS-ON

```
// Recommended: constructor injection
@Service
public class OrderService {
    private final PaymentGateway gateway;
    public OrderService(PaymentGateway gateway) { // no @Autowired needed
        this.gateway = gateway;                   // if only one constructor
    }
}

// Discouraged: field injection
@Service
public class OrderServiceLegacy {
    @Autowired
    private PaymentGateway gateway; // can't be final, hard to unit test
}
```

Interview angle: Spring's own docs recommend constructor injection as the default; field injection is considered an anti-pattern for anything beyond quick prototypes.

Chapter 3 — Spring MVC

11 What is the role of @Controller and @RestController?

EXPLANATION

`@Controller` marks a class as a Spring MVC handler whose methods, by default, return **view names** resolved by a `ViewResolver` (e.g. Thymeleaf templates). `@RestController` is a meta-annotation combining `@Controller` + `@ResponseBody`, meaning every method's return value is serialized directly into the HTTP response body (typically JSON) instead of being resolved as a view — the standard choice for building REST APIs.

HANDS-ON

```
@Controller
public class WebController {
    @GetMapping("/greeting")
    public String greeting(Model model) {
        model.addAttribute("name", "World");
        return "greeting"; // resolves to templates/greeting.html
    }
}

@RestController
public class ApiController {
    @GetMapping("/api/greeting")
    public Map<String, String> greeting() {
        return Map.of("message", "Hello, World"); // serialized straight to JSON
    }
}
```

12 Difference between @RequestParam and @PathVariable?

EXPLANATION

`@PathVariable` extracts a value embedded in the URI path (typically identifying a resource), while `@RequestParam` extracts a value from the query string or form data (typically filtering/paging options).

HANDS-ON

```
// GET /users/42?includeOrders=true
@GetMapping("/users/{id}")
public User getUser(
        @PathVariable Long id,                                  // 42
        @RequestParam(defaultValue = "false") boolean includeOrders) {
    return userService.find(id, includeOrders);
}
```

> Interview angle: use path variables for resource identity (RESTful design) and query params for optional filters/sorting/pagination.

13 How does @RequestBody work?

EXPLANATION

`@RequestBody` tells Spring MVC to deserialize the raw HTTP request body into a Java object, using an `HttpMessageConverter` (Jackson's `MappingJackson2HttpMessageConverter` by default for JSON). Bean Validation annotations combined with `@Valid` trigger validation right after binding.

HANDS-ON

```
public record CreateUserRequest(
    @NotBlank String name,
    @Email String email) {}

@PostMapping("/users")
public ResponseEntity<User> createUser(@Valid @RequestBody CreateUserRequest req) {
    User saved = userService.create(req.name(), req.email());
    return ResponseEntity.status(HttpStatus.CREATED).body(saved);
}
// POST /users  { "name": "Ana", "email": "ana@example.com" }
// Invalid payload -> Spring throws MethodArgumentNotValidException -> 400
```

14 What is the use of @ModelAttribute?

EXPLANATION

`@ModelAttribute` serves two purposes: (1) on a method parameter, it binds request parameters/form fields to a Java object (used heavily with HTML forms, unlike `@RequestBody` which expects JSON); (2) on a method in a controller, it pre-populates data into the `Model` for every request handler in that controller.

HANDS-ON

```
@Controller
public class RegistrationController {

    // Runs before every @RequestMapping method in this controller,
    // adding "countries" to the model automatically
    @ModelAttribute("countries")
    public List<String> populateCountries() {
        return List.of("USA", "India", "Germany");
    }

    // Binds form fields name=&email=&country= into a RegistrationForm
    @PostMapping("/register")
    public String register(@ModelAttribute RegistrationForm form) {
        userService.register(form);
        return "redirect:/success";
    }
}
```

Chapter 4 — Spring Data JPA

15 What is Spring Data JPA?

EXPLANATION

Spring Data JPA is an abstraction layer over JPA/Hibernate that eliminates boilerplate DAO code. By defining an interface that extends `JpaRepository`, you get CRUD, pagination, and sorting for free, plus the ability to derive queries from method names or write custom JPQL/native SQL declaratively.

HANDS-ON

```
@Entity
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private BigDecimal price;
    // getters/setters
}

public interface ProductRepository extends JpaRepository<Product, Long> {
    // no implementation needed - Spring generates it at runtime
}

@Service
public class ProductService {
    private final ProductRepository repo;
    ProductService(ProductRepository repo) { this.repo = repo; }

    public Product save(Product p) { return repo.save(p); }
    public List<Product> all() { return repo.findAll(); }
}
```

16 Difference between CrudRepository and JpaRepository?

EXPLANATION

Interface	Provides
`CrudRepository<T, ID>`	Basic CRUD: `save`, `findById`, `findAll`, `deleteById`, `count`
`PagingAndSortingRepository<T, ID>`	Extends CrudRepository, adds `findAll(Pageable)` / `findAll(Sort)`
`JpaRepository<T, ID>`	Extends both above, adds JPA-specific batch ops: `saveAll`, `flush`, `deleteInBatch`, and returns `List` instead of `Iterable`

HANDS-ON

```
public interface ProductRepository extends JpaRepository<Product, Long> {
    Page<Product> findByPriceGreaterThan(BigDecimal price, Pageable pageable);
}

Page<Product> page = productRepository.findByPriceGreaterThan(
        new BigDecimal("50"), PageRequest.of(0, 10, Sort.by("name")));
```

Interview angle: default to `JpaRepository` in real projects — it's a strict superset, so there's rarely a reason to pick the narrower interfaces unless you're deliberately hiding JPA-specific methods from a layer.

17 What are derived query methods?

EXPLANATION

Spring Data parses repository method names following a keyword convention (`findBy`, `countBy`, `existsBy`, combined with `And` / `Or` / `GreaterThan` / `Containing` / `OrderBy`, etc.) and generates the correct JPQL query automatically at startup — no implementation code, no annotations required.

HANDS-ON

```
public interface ProductRepository extends JpaRepository<Product, Long> {

    List<Product> findByNameContainingIgnoreCase(String keyword);

    List<Product> findByPriceBetweenOrderByPriceAsc(BigDecimal min, BigDecimal max);

    Optional<Product> findFirstByNameOrderByPriceDesc(String name);

    boolean existsByNameAndPriceLessThan(String name, BigDecimal price);

    long countByPriceGreaterThan(BigDecimal price);
}
```

> Interview angle: for anything beyond ~3 conditions, prefer `@Query` with JPQL for readability — derived names get unwieldy fast.

18 What does the @Entity annotation do?

EXPLANATION

`@Entity` marks a POJO as a JPA-managed persistent class, mapping it to a database table. The JPA provider (Hibernate) scans it for metadata — field-to-column mapping, primary key, relationships — and manages its lifecycle (transient, managed, detached, removed) within a persistence context.

HANDS-ON

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "product_name", nullable = false, length = 120)
    private String name;

    @Column(precision = 10, scale = 2)
    private BigDecimal price;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id")
    private Category category;
}
```

19 Use of @Id and @GeneratedValue annotations?

EXPLANATION

`@Id` designates the primary key field of an entity. `@GeneratedValue` configures how that key is generated, via a `strategy`:

Strategy	Behavior
`IDENTITY`	Delegates to DB auto-increment column
`SEQUENCE`	Uses a DB sequence object (preferred with PostgreSQL/Oracle, supports batching)
`TABLE`	Simulates a sequence using a table — portable but slower
`AUTO`	Provider picks a strategy based on the dialect

HANDS-ON

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "product_seq")
@SequenceGenerator(name = "product_seq", sequenceName = "product_id_seq", allocationSize = 50)
private Long id;
```

> Interview angle: `IDENTITY` disables JDBC batch inserts in Hibernate because the ID must be known immediately after each insert — a common performance gotcha.

Chapter 5 — Hibernate

20 What is Hibernate and how is it related to JPA?

EXPLANATION

JPA (Jakarta Persistence API) is a **specification** — a set of interfaces and annotations describing how Java objects should map to relational tables. Hibernate is the most widely used **implementation** of that specification (an ORM engine). Spring Boot's `spring-boot-starter-data-jpa` pulls in Hibernate as the default JPA provider. You code against JPA interfaces (`EntityManager`, `@Entity`), and Hibernate does the actual SQL generation, caching, and dirty-checking underneath.

HANDS-ON

```
# application.properties - Hibernate-specific tuning still available
# even though the app is coded against the JPA API
spring.jpa.properties.hibernate.show_sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.hibernate.ddl-auto=update
```

Interview angle: this is why you could theoretically swap Hibernate for EclipseLink without changing your `@Entity` / `@Repository` code — the abstraction is the point.

21 What is lazy loading and eager loading?

EXPLANATION

These control **when** associated entities are fetched from the database relative to their parent. `LAZY` defers loading an association until it's actually accessed (via a proxy), reducing unnecessary queries. `EAGER` loads the association immediately alongside the parent, in the same or an additional query. Default fetch types: `@OneToMany` / `@ManyToMany` are LAZY; `@ManyToOne` / `@OneToOne` are EAGER by default (a common gotcha).

HANDS-ON

```
@Entity
public class Order {
    @Id @GeneratedValue
    private Long id;

    @OneToMany(mappedBy = "order", fetch = FetchType.LAZY)
    private List<OrderItem> items; // loaded only when .getItems() is called

    @ManyToOne(fetch = FetchType.LAZY) // explicitly override default EAGER
    @JoinColumn(name = "customer_id")
    private Customer customer;
}

// Accessing items outside a transaction/session throws
// LazyInitializationException unless fetched within scope:
@Transactional(readOnly = true)
public Order getOrderWithItems(Long id) {
    Order order = orderRepo.findById(id).orElseThrow();
    order.getItems().size(); // forces initialization while session is open
    return order;
}
```

Interview angle: always default to LAZY for collections and use JOIN FETCH or entity graphs when you need eager loading for a specific query, rather than making the mapping globally EAGER.

22 Difference between save() and saveAndFlush()?

EXPLANATION

`save()` persists/updates an entity but the actual SQL `INSERT` / `UPDATE` may be deferred until the persistence context is flushed (e.g. at transaction commit, or before a query that needs consistent state). `saveAndFlush()` immediately forces a flush, synchronously writing to the database and making the change visible to any subsequent native query within the same transaction.

HANDS-ON

```
@Transactional
public void demonstrateFlush() {
    Product p = new Product("Widget", new BigDecimal("9.99"));
    productRepository.save(p);
    // SQL insert may not have executed yet - still buffered

    productRepository.saveAndFlush(p);
    // SQL insert has definitely executed now - useful right before
    // a native/JDBC query that needs to see this row
}
```

Interview angle: overusing `saveAndFlush` hurts performance by defeating Hibernate's write-behind batching — use it only when you specifically need immediate consistency.

23 What is the N+1 query problem?

EXPLANATION

The N+1 problem happens when fetching a list of N parent entities triggers 1 query for the parents, then N additional queries — one per parent — to lazily load each one's association. This silently devastates performance as data grows.

HANDS-ON

```
// PROBLEM: 1 query for orders, then N queries, one per order, for items
List<Order> orders = orderRepository.findAll();
for (Order o : orders) {
    System.out.println(o.getItems().size()); // triggers a query each time
}

// FIX 1: JOIN FETCH - one query total
@Query("SELECT DISTINCT o FROM Order o LEFT JOIN FETCH o.items")
List<Order> findAllWithItems();

// FIX 2: @EntityGraph - declarative alternative to JOIN FETCH
@EntityGraph(attributePaths = "items")
@Query("SELECT o FROM Order o")
List<Order> findAllWithItemsGraph();

// FIX 3: batch fetching, reduces N to N/batchSize
@BatchSize(size = 20)
private List<OrderItem> items;
```

Interview angle: mention enabling `spring.jpa.properties.hibernate.generate_statistics=true` or a tool like p6spy/Hibernate SQL logging in dev to actually spot N+1 patterns before they hit production.

Chapter 6 — Spring Security

24 What is Spring Security?

EXPLANATION

Spring Security is a highly customizable framework providing authentication (who are you) and authorization (what can you do) for Spring applications, plus protections against common attacks (CSRF, session fixation, clickjacking). It integrates with the servlet filter chain, intercepting every request before it reaches your controllers.

HANDS-ON

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>

// Adding this dependency alone secures ALL endpoints by default
// with a generated password logged at startup - proving Security
// is active out of the box before any custom config is written
```

25 How do you secure endpoints in Spring Boot?

EXPLANATION

Endpoint security is configured via a `SecurityFilterChain` bean (the modern, lambda-DSL approach replacing the older `WebSecurityConfigurerAdapter`), specifying which paths require authentication, which roles they require, and how authentication happens (form login, HTTP Basic, JWT, OAuth2).

HANDS-ON

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
                .anyRequest().authenticated()
            )
            .csrf(csrf -> csrf.disable()) // typical for stateless REST APIs
            .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

26 Difference between authentication and authorization?

EXPLANATION

	Authentication	Authorization
Question answered	"Who are you?"	"What are you allowed to do?"
When it happens	Login / credential check	After identity is established, per request/resource
Spring Security mechanism	`AuthenticationManager`, `AuthenticationProvider`	`AccessDecisionManager` / `authorizeHttpRequests`, `@PreAuthorize`

HANDS-ON

```
// Authentication: verifying username/password produces an Authentication object
Authentication auth = authenticationManager.authenticate(
    new UsernamePasswordAuthenticationToken(username, password));

// Authorization: once authenticated, checking role/permission per-method
@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/products/{id}")
public void deleteProduct(@PathVariable Long id) { productService.delete(id); }
```

27 What is the purpose of UserDetailsService?

EXPLANATION

`UserDetailsService` is the core interface Spring Security uses to load user-specific data (username, password hash, authorities, account status) during authentication. You implement its single method, `loadUserByUsername`, to plug in your own data source — typically a database via a repository.

HANDS-ON

```
@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;
    CustomUserDetailsService(UserRepository repo) { this.userRepository = repo; }

    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found: " + username));

        return org.springframework.security.core.userdetails.User
            .withUsername(user.getUsername())
            .password(user.getPasswordHash())
            .authorities(user.getRoles().toArray(new String[0]))
            .build();
    }
}
```

28 What are roles and authorities in Spring Security?

EXPLANATION

An **authority** is a granular permission granted to a principal (e.g. `READ_PRODUCTS`), represented as `GrantedAuthority`. A **role** is a special-cased authority conventionally prefixed `ROLE_` (e.g. `ROLE_ADMIN`) representing a broader category a user belongs to. Methods like `hasRole("ADMIN")` automatically add the `ROLE_` prefix under the hood, whereas `hasAuthority("ROLE_ADMIN")` requires the exact string.

HANDS-ON

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/admin/**").hasRole("ADMIN")            // checks ROLE_ADMIN
    .requestMatchers("/reports/**").hasAuthority("REPORT_VIEW") // exact match
    .requestMatchers("/manager/**").hasAnyRole("ADMIN", "MANAGER")
)

@PreAuthorize("hasAuthority('REPORT_VIEW') and hasRole('MANAGER')")
public Report generateReport() { ... }
```

> Interview angle: roles are just a naming convention layered on top of authorities — under the hood, Spring Security only understands `GrantedAuthority` objects.

Chapter 7 — REST APIs

29 What is REST and why is it used?

EXPLANATION

REST (Representational State Transfer) is an architectural style for networked applications built around stateless client-server communication, resources identified by URIs, and a uniform interface using standard HTTP verbs. It's used because it's simple, scales horizontally (no server-side session state), leverages HTTP's existing infrastructure (caching, status codes), and is language/platform agnostic — any client that speaks HTTP/JSON can consume it.

HANDS-ON

```
@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService service;
    ProductController(ProductService service) { this.service = service; }

    @GetMapping                     // GET  /api/products
    public List<Product> all() { return service.findAll(); }

    @GetMapping("/{id}")            // GET  /api/products/1
    public Product one(@PathVariable Long id) { return service.find(id); }

    @PostMapping                    // POST /api/products
    public Product create(@RequestBody Product p) { return service.save(p); }
}
```

30 What are the common HTTP methods in REST?

EXPLANATION

Method	Purpose	Idempotent?	Safe?
GET	Retrieve a resource	Yes	Yes
POST	Create a resource	No	No
PUT	Replace a resource entirely	Yes	No
PATCH	Partially update a resource	No (typically)	No
DELETE	Remove a resource	Yes	No

HANDS-ON

```
@PutMapping("/{id}")
public Product replace(@PathVariable Long id, @RequestBody Product p) {
    return service.replace(id, p); // full replacement
}

@PatchMapping("/{id}")
public Product update(@PathVariable Long id, @RequestBody Map<String,Object> fields) {
    return service.applyPatch(id, fields); // partial update
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    service.delete(id);
    return ResponseEntity.noContent().build();
}
```

31 Difference between PUT and PATCH?

EXPLANATION

`PUT` replaces the **entire** resource with the payload provided — any field omitted is typically reset/nulled. `PATCH` applies a **partial** modification, updating only the fields included in the request, leaving the rest untouched.

HANDS-ON

```
// PUT /products/5  { "name": "Widget", "price": 12.99 }
// -> replaces the whole Product row; any field not sent may become null

// PATCH /products/5  { "price": 14.99 }
// -> updates only price; name and everything else stays as-is
@PatchMapping("/{id}")
public Product patch(@PathVariable Long id, @RequestBody Map<String, Object> updates) {
    Product existing = service.find(id);
    updates.forEach((key, value) -> {
        switch (key) {
            case "price" -> existing.setPrice(new BigDecimal(value.toString()));
            case "name"  -> existing.setName((String) value);
        }
    });
    return service.save(existing);
}
```

32 What is ResponseEntity used for?

EXPLANATION

`ResponseEntity<T>` gives full control over the HTTP response — status code, headers, and body — rather than Spring inferring a default 200 OK. It's the standard way to return proper REST semantics (201 Created with a Location header, 404 Not Found, 204 No Content, etc.).

HANDS-ON

```
@PostMapping
public ResponseEntity<Product> create(@RequestBody Product p) {
    Product saved = service.save(p);
    URI location = URI.create("/api/products/" + saved.getId());
    return ResponseEntity.created(location).body(saved); // 201 + Location header
}

@GetMapping("/{id}")
public ResponseEntity<Product> get(@PathVariable Long id) {
    return service.findOptional(id)
        .map(ResponseEntity::ok)                 // 200
        .orElse(ResponseEntity.notFound().build()); // 404
}
```

33 How do you handle API versioning?

EXPLANATION

API versioning lets you evolve an API without breaking existing clients. Common strategies: **URI versioning** (`/api/v1/...`), **query parameter** (`?version=1`), **custom header** (`X-API-Version: 1`), and **media-type/content-negotiation** versioning (`Accept: application/vnd.company.v1+json`).

HANDS-ON

```
// URI versioning - simplest, most common in practice
@RestController
@RequestMapping("/api/v1/products")
public class ProductControllerV1 { /* legacy shape */ }

@RestController
@RequestMapping("/api/v2/products")
public class ProductControllerV2 { /* new shape, e.g. paginated */ }

// Header-based versioning
@GetMapping(value = "/products", headers = "X-API-Version=2")
public List<ProductV2> getV2() { ... }
```

> Interview angle: URI versioning is easiest to document and cache but "pollutes" the URI; header/media-type versioning is more RESTfully "pure" but harder for clients to discover and test manually.

Chapter 8 — JWT

34 What is JWT?

EXPLANATION

JSON Web Token (JWT) is a compact, self-contained, URL-safe token format for securely transmitting claims (statements about a user) between parties. It's digitally signed (HMAC or RSA/ECDSA), so the receiving server can verify it hasn't been tampered with — without needing to store session state.

HANDS-ON

```
// Generating a JWT (using jjwt library)
String token = Jwts.builder()
    .setSubject("john.doe")
    .claim("roles", List.of("USER"))
    .setIssuedAt(new Date())
    .setExpiration(new Date(System.currentTimeMillis() + 3_600_000))
    .signWith(secretKey, SignatureAlgorithm.HS256)
    .compact();

// Result looks like:
// eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJqb2huLmRvZSIsInJvbGVzIjpbIlVTRVIiXX0.4f_9cS...
```

35 What are the three parts of a JWT?

EXPLANATION

Part	Content	Encoding
Header	Token type (JWT) + signing algorithm (e.g. HS256)	Base64Url of JSON
Payload	Claims: subject, roles, expiration, issuer, custom data	Base64Url of JSON
Signature	HMAC/RSA signature of header+payload, using a secret/private key	Base64Url

HANDS-ON

```
// Header (decoded):    {"alg":"HS256","typ":"JWT"}
// Payload (decoded):   {"sub":"john.doe","roles":["USER"],"exp":1731000000}
// Signature: HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)

// Joined with dots: header.payload.signature
// Decoding a token (verification) in Java:
Claims claims = Jwts.parserBuilder()
    .setSigningKey(secretKey)
    .build()
    .parseClaimsJws(token)
    .getBody();
System.out.println(claims.getSubject()); // john.doe
```

> Interview angle: the payload is only Base64-encoded, not encrypted — never put secrets/passwords in JWT claims.

36 How is JWT used in Spring Boot authentication?

EXPLANATION

The typical flow: (1) client POSTs credentials to a login endpoint, (2) server verifies them and issues a signed JWT, (3) client stores the token and sends it as `Authorization: Bearer <token>` on subsequent requests, (4) a custom `OncePerRequestFilter` intercepts each request, validates the token, and populates the `SecurityContext` so downstream authorization checks work normally.

HANDS-ON

```
public class JwtAuthFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                    FilterChain chain) throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            if (jwtService.isValid(token)) {
                String username = jwtService.extractUsername(token);
                UserDetails user = userDetailsService.loadUserByUsername(username);
                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(user, null, user.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}

// Registered before the default filter in SecurityConfig:
http.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
```

37 Difference between session-based auth and JWT auth?

EXPLANATION

Aspect	Session-based	JWT-based
State	Stateful — server stores session data	Stateless — all claims travel in the token
Scalability	Needs sticky sessions or shared session store (Redis)	Scales horizontally with no shared state
Revocation	Easy — delete session server-side	Hard — must use short expiry + blacklist/refresh-token strategy
Storage	Session ID in a cookie	Token in header, local storage, or cookie
Typical use	Traditional server-rendered web apps	REST APIs, microservices, mobile/SPA clients

HANDS-ON

```
// Session-based: server enforces statefulness
http.sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.ALWAYS));

// JWT-based: server declares itself stateless - no HttpSession used at all
http.sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
```

> Interview angle: mention refresh tokens as the standard mitigation for JWT's revocation weakness — short-lived access tokens + a longer-lived refresh token that can be revoked server-side.

Chapter 9 — Microservices

38 What are microservices?

EXPLANATION

Microservices are an architectural style where an application is decomposed into small, independently deployable services, each owning a single business capability and its own data store, communicating over the network (HTTP/REST, gRPC, messaging). This contrasts with a monolith, where all functionality ships and scales as one unit.

HANDS-ON

```
// Instead of one monolithic app, you'd have separate deployable Spring Boot apps:
// order-service      (port 8081) - owns Order data, exposes /orders
// inventory-service  (port 8082) - owns Stock data, exposes /inventory
// payment-service    (port 8083) - owns Payment data, exposes /payments

// order-service calling inventory-service over HTTP:
@Service
public class InventoryClient {
    private final RestClient restClient;
    public boolean checkStock(Long productId) {
        return restClient.get()
            .uri("http://inventory-service/inventory/{id}", productId)
            .retrieve()
            .body(StockResponse.class)
            .available();
    }
}
```

39 What is service discovery?

EXPLANATION

In a microservices system, instances scale up/down and their network locations change dynamically. Service discovery lets services find each other by logical name instead of hardcoded host:port. A registry (e.g. Netflix Eureka, Consul) keeps track of healthy instances; clients or a load balancer query the registry to resolve a service name to an actual address.

HANDS-ON

```
// Eureka server (registry) - separate Spring Boot app
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServerApplication { }

// Each microservice registers itself as a Eureka client
@SpringBootApplication
@EnableDiscoveryClient
public class InventoryServiceApplication { }

# application.properties for inventory-service
spring.application.name=inventory-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

// Calling another service by logical name (load-balanced automatically)
@LoadBalanced
@Bean
public RestClient.Builder restClientBuilder() { return RestClient.builder(); }
// then: restClient.get().uri("http://inventory-service/inventory/1")...
```

40 What is the role of an API Gateway?

EXPLANATION

An API Gateway is a single entry point that sits in front of all microservices, handling cross-cutting concerns centrally: routing requests to the correct service, authentication/authorization, rate limiting, request/response transformation, and aggregating responses from multiple services. It shields clients from needing to know internal service topology.

HANDS-ON

```
# Spring Cloud Gateway route configuration
spring.cloud.gateway.routes[0].id=order-service
spring.cloud.gateway.routes[0].uri=lb://order-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/api/orders/**

spring.cloud.gateway.routes[1].id=inventory-service
spring.cloud.gateway.routes[1].uri=lb://inventory-service
spring.cloud.gateway.routes[1].predicates[0]=Path=/api/inventory/**

// Programmatic route with a filter (e.g. adding an auth header)
@Bean
public RouteLocator routes(RouteLocatorBuilder builder) {
    return builder.routes()
        .route("orders", r -> r.path("/api/orders/**")
            .filters(f -> f.stripPrefix(1))
            .uri("lb://order-service"))
        .build();
}
```

41 How does Spring Boot support microservices?

EXPLANATION

Spring Boot itself provides the fast, self-contained, independently-runnable service units (embedded server, externalized config, Actuator health checks per instance). The broader **Spring Cloud** project builds on top of it to provide the missing distributed-systems pieces: service discovery (Eureka/Consul), gateway routing (Spring Cloud Gateway), client-side load balancing, centralized configuration (Spring Cloud Config), circuit breakers (Resilience4j), and distributed tracing (Micrometer Tracing/Zipkin).

HANDS-ON

```
// Circuit breaker with Resilience4j to prevent cascading failures
@CircuitBreaker(name = "inventoryService", fallbackMethod = "fallbackStock")
public StockResponse checkStock(Long productId) {
    return inventoryClient.getStock(productId);
}

public StockResponse fallbackStock(Long productId, Throwable t) {
    return new StockResponse(productId, false); // degrade gracefully
}

// Centralized config - all services pull config from one place
spring.config.import=optional:configserver:http://localhost:8888
```

> Interview angle: be ready to explain that Spring Boot = the service runtime, Spring Cloud = the glue that makes many such services work together reliably.

Chapter 10 — Exception Handling

42 What is @ControllerAdvice used for?

EXPLANATION

`@ControllerAdvice` defines a global, cross-cutting component applied to every `@Controller` / `@RestController` in the application — most commonly used to centralize exception handling (via `@ExceptionHandler` methods), but also supports global `@ModelAttribute` and `@InitBinder` logic. It avoids duplicating try/catch blocks in every controller.

HANDS-ON

```
@RestControllerAdvice // = @ControllerAdvice + @ResponseBody
public class GlobalExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ProductNotFoundException ex) {
        ErrorResponse body = new ErrorResponse("NOT_FOUND", ex.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        ErrorResponse body = new ErrorResponse("INTERNAL_ERROR", "Something went wrong");
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(body);
    }
}
```

43 Difference between @ExceptionHandler and global exception handling?

EXPLANATION

An `@ExceptionHandler` method declared directly inside a single `@Controller` only catches exceptions thrown by handlers **in that one controller** — it's local/scoped. Placing `@ExceptionHandler` methods inside a class annotated `@ControllerAdvice` makes them **global** — applied across all controllers in the application, giving one centralized place for consistent error responses.

HANDS-ON

```
// LOCAL - only catches exceptions from THIS controller
@RestController
public class ProductController {
    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<String> handle(ProductNotFoundException ex) {
        return ResponseEntity.status(404).body(ex.getMessage());
    }
}

// GLOBAL - catches the same exception from ANY controller
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<String> handle(ProductNotFoundException ex) {
        return ResponseEntity.status(404).body(ex.getMessage());
    }
}
```

> Interview angle: if both exist for the same exception type, the more specific (local, controller-level) handler wins over the global one.

44 How do you create a custom exception response?

EXPLANATION

A custom exception response standardizes the JSON shape of every error returned by the API (error code, message, timestamp, path, validation details) — critical for consistent client-side error handling. You define a custom exception class, a response DTO, and wire them together through `@ControllerAdvice`.

HANDS-ON

```
public class ProductNotFoundException extends RuntimeException {
    public ProductNotFoundException(Long id) {
        super("Product not found with id: " + id);
    }
}

public record ErrorResponse(
    String errorCode, String message, Instant timestamp, String path) {}

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
            ProductNotFoundException ex, HttpServletRequest req) {
        ErrorResponse body = new ErrorResponse(
            "PRODUCT_NOT_FOUND", ex.getMessage(), Instant.now(), req.getRequestURI());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors()
          .forEach(err -> errors.put(err.getField(), err.getDefaultMessage()));
        return ResponseEntity.badRequest().body(errors);
    }
}
```

Chapter 11 — Transactions

45 What is the @Transactional annotation?

EXPLANATION

`@Transactional` declaratively wraps a method in a database transaction: Spring creates a proxy around the bean that begins a transaction before the method runs and commits it on success, or rolls it back on a runtime exception. This replaces manually calling `begin()` / `commit()` / `rollback()`.

HANDS-ON

```
@Service
public class OrderService {

    @Transactional
    public void placeOrder(Order order, List<OrderItem> items) {
        orderRepository.save(order);
        for (OrderItem item : items) {
            inventoryService.decrementStock(item.getProductId(), item.getQuantity());
        }
        // If decrementStock throws anywhere, the order.save() above
        // is rolled back too - all-or-nothing
    }
}
```

Interview angle: `@Transactional` only works through a Spring-managed proxy — calling an `@Transactional` method from within the same class (self-invocation) bypasses the proxy and silently skips the transaction.

46 What is transaction propagation?

EXPLANATION

Propagation defines how a transactional method behaves when called from within an already-active transaction — whether it joins the existing one, suspends it, or starts a nested one.

Propagation	Behavior
`REQUIRED` (default)	Joins existing transaction, or creates a new one if none exists
`REQUIRES_NEW`	Always suspends any existing transaction and starts a fresh, independent one
`NESTED`	Runs within a savepoint of the outer transaction — can roll back independently
`MANDATORY`	Requires an existing transaction; throws if none is active
`NOT_SUPPORTED`	Suspends any existing transaction and runs non-transactionally

HANDS-ON

```
@Service
public class AuditService {

    // Always commits independently, even if the calling transaction rolls back -
    // useful for audit logs that must survive a failed business operation
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void logAction(String action) {
        auditRepository.save(new AuditEntry(action, Instant.now()));
    }
}

@Service
public class OrderService {
    @Transactional
    public void placeOrder(Order order) {
        orderRepository.save(order);
        auditService.logAction("ORDER_PLACED"); // commits even if this method later fails
        throw new RuntimeException("payment failed"); // rolls back the order, not the audit log
    }
}
```

47 What is rollback in Spring transactions?

EXPLANATION

Rollback undoes all database changes made during a transaction when it fails, restoring data to its pre-transaction state. By default, Spring rolls back only on **unchecked** exceptions (`RuntimeException` and `Error`) — checked exceptions do **not** trigger a rollback unless explicitly configured.

HANDS-ON

```
@Transactional
public void transferFunds(Account from, Account to, BigDecimal amount) {
    from.debit(amount);
    accountRepository.save(from);

    if (amount.compareTo(BigDecimal.valueOf(1_000_000)) > 0) {
        throw new IllegalStateException("Amount exceeds limit"); // unchecked -> rolls back debit too
    }

    to.credit(amount);
    accountRepository.save(to);
}

// Forcing rollback on a CHECKED exception - opt-in required
@Transactional(rollbackFor = InsufficientFundsException.class)
public void withdraw(Account acc, BigDecimal amount) throws InsufficientFundsException {
    if (acc.getBalance().compareTo(amount) < 0) {
        throw new InsufficientFundsException("Not enough balance");
    }
    acc.debit(amount);
}

// Excluding a runtime exception from triggering rollback
@Transactional(noRollbackFor = ValidationWarning.class)
public void save(Entity e) { ... }
```

Interview angle: this default (checked exceptions don't roll back) trips up many developers coming from JTA/EJB backgrounds — always double check with `rollbackFor` when using checked exceptions in a transactional method.

Chapter 12 — Caching

48 What is caching in Spring Boot?

EXPLANATION

Caching stores the results of expensive operations (DB queries, external API calls) in a fast-access store (in-memory or distributed like Redis) so subsequent identical requests are served instantly without redoing the work. Spring's caching abstraction (`@EnableCaching` + `@Cacheable` / `@CachePut` / `@CacheEvict`) provides a provider-agnostic API — the underlying cache can be a simple `ConcurrentHashMap`, Caffeine, Ehcache, or Redis, swapped without changing business code.

HANDS-ON

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-cache</artifactId>
</dependency>

@SpringBootApplication
@EnableCaching
public class DemoApplication { }
```

49 What is the use of @Cacheable?

EXPLANATION

`@Cacheable` wraps a method so Spring checks the cache first using a key derived from the method arguments; on a cache hit, the method body is skipped entirely and the cached value is returned. On a miss, the method executes normally and its return value is stored under that key for next time.

HANDS-ON

```
@Service
public class ProductService {

    @Cacheable(value = "products", key = "#id")
    public Product findById(Long id) {
        System.out.println("Hitting the database for id " + id);
        return productRepository.findById(id)
            .orElseThrow(() -> new ProductNotFoundException(id));
    }
}

// First call: prints "Hitting the database..." and caches the result
Product p1 = productService.findById(1L);
// Second call with the same id: returns instantly from cache, no print, no DB hit
Product p2 = productService.findById(1L);
```

50 Difference between @Cacheable, @CachePut, and @CacheEvict?

EXPLANATION

Annotation	Behavior	When to use
`@Cacheable`	Checks cache first; skips method on hit	Read operations (lookups)
`@CachePut`	Always runs the method, then updates the cache with the new result	After an update, to keep cache in sync without skipping the write
`@CacheEvict`	Removes an entry (or all entries) from the cache	After a delete, or to force the next read to recompute

HANDS-ON

```
@Service
public class ProductService {

    @Cacheable(value = "products", key = "#id")
    public Product findById(Long id) { return productRepository.findById(id).orElseThrow(); }

    @CachePut(value = "products", key = "#product.id")
    public Product update(Product product) {
        return productRepository.save(product); // always executes, refreshes cache entry
    }

    @CacheEvict(value = "products", key = "#id")
    public void delete(Long id) {
        productRepository.deleteById(id); // removes the now-stale cache entry
    }

    @CacheEvict(value = "products", allEntries = true)
    public void clearAllCache() { } // e.g. scheduled nightly cache refresh
}
```

Interview angle: mixing these correctly around writes is the classic pitfall — forgetting `@CacheEvict` / `@CachePut` after an update leaves stale data being served from `@Cacheable` reads.